Sprint -2 [Writing knowledge]

SPRINT 2 — AI WRITING ASSISTANT + AI KNOWLEDGE SEARCH

Features 1 and 2 — Full Implementation

PREREQUISITE: Sprint 1 must be complete and verified

PASTE THIS INTO REPLIT AI AGENT

WHAT TO BUILD IN THIS SPRINT

1. Feature 1: AI Writing Assistant (all 4 triggers + role-based writing + brand voice)
2. Feature 2: AI Knowledge Search (universal search UI + Ask AI mode + proactive suggestions)

After this sprint, verify:

- Press Space on empty editor line → AI prompt bar appears → type request → AI streams response into editor
- Select text → click AI button → choose "Improve" → see diff view → Accept/Reject
- Press Cmd+J → AI prompt bar opens → ask about workspace data → get answer with citations
- Press Cmd+K → universal search → type keyword → see results from pages/tasks/messages

- Switch to "Ask AI" tab → type natural language question → get AI answer with sources

---

FEATURE 1: AI WRITING ASSISTANT

1.1 tRPC Router

Create file: `src/server/routers/ai-writing.ts`

```
// src/server/routers/ai-writing.ts

import { z } from "zod";
import { router, protectedProcedure } from "@/lib/trpc";
import { aiService } from "@/lib/ai/ai-service";
import { logAiUsage } from "@/server/services/ai-usage-logge
r";
import { prisma } from "@/lib/prisma";
import { observable } from "@trpc/server/observable";

// ============================================================
==
// INPUT SCHEMAS
// ============================================================
==

const writeActionSchema = z.enum([
  "write", "improve", "shorten", "lengthen", "tone", "gramma
r",
  "translate", "summarize", "brainstorm", "outline", "continu
e",
  "explain", "action_items", "custom",
]);

const toneSchema = z.enum([
  "professional", "casual", "friendly", "academic", "persuasi
ve",
```

```
]);

const writerRoleSchema = z.enum([
  "general", "marketer", "engineer", "executive", "designer",
"sales", "support", "hr",
]);

// ============================================================
==
// SYSTEM PROMPTS (one per action)
// ============================================================
==

const ACTION_PROMPTS: Record<string, string> = {
  write: "You are an expert writer. Generate clear, well-stru
ctured content based on the user's request. Write in a natura
l, engaging style.",
  improve: "You are an expert editor. Improve the following t
ext to be clearer, more professional, and better structured.
Keep the same meaning but make it significantly better.",
  shorten: "You are an expert editor. Condense the following
text to be shorter while keeping all key information. Target
50-60% of the original length.",
  lengthen: "You are an expert writer. Expand the following t
ext with more detail, examples, and explanation. Target 150-2
00% of the original length.",
  tone: "You are an expert editor. Rewrite the following text
in the specified tone while keeping the same meaning and info
rmation.",
  grammar: "You are a grammar expert. Fix all spelling, gramm
ar, and punctuation errors. Make minimal changes — only fix e
rrors, do not change style or meaning.",
  translate: "You are an expert translator. Translate the fol
lowing text accurately into the specified language. Preserve
formatting and tone.",
  summarize: "You are an expert summarizer. Create a clear, c
```

```
oncise summary of the following text in 2-4 sentences. Captur
e the key points.",
  brainstorm: "You are a creative brainstorming expert. Gener
ate 5-10 unique, practical ideas based on the topic. Each ide
a should be one sentence with a brief explanation.",
  outline: "You are a document structuring expert. Create a c
lear, hierarchical outline with H2 headings and bullet points
for the given topic.",
  continue: "You are an expert writer. Continue writing from
where the text ends. Match the existing style, tone, and topi
c seamlessly.",
  explain: "You are an expert explainer. Provide a clear, eas
y-to-understand explanation of the following text. Use simple
language and examples if helpful.",
  action_items: "You are a task extraction expert. Extract al
l action items, tasks, and to-dos from the following text. Fo
rmat each as a clear, actionable checkbox item.",
  custom: "You are an expert AI assistant helping with writin
g and editing.",
};

const ROLE_PROMPTS: Record<string, string> = {
  general: "",
  marketer: "Write in a persuasive, benefit-focused style. Us
e strong calls-to-action. Focus on value propositions and cus
tomer impact.",
  engineer: "Write technically precise content. Use proper te
chnical terminology. Include examples and edge cases where re
levant.",
  executive: "Write concise, strategic content. Focus on busi
ness impact, decisions, and outcomes. Use data when availabl
e.",
  designer: "Write with attention to user experience. Focus o
n usability, aesthetics, and user journey. Use visual languag
e.",
  sales: "Write in a convincing, relationship-focused style.
```

```
Highlight benefits, social proof, and urgency. Be warm but pr
ofessional.",
  support: "Write in a helpful, empathetic style. Be clear an
d solution-oriented. Anticipate follow-up questions.",
  hr: "Write in an inclusive, professional style. Be clear ab
out policies and expectations. Use respectful, people-first l
anguage.",
};

function buildSystemPrompt(
  action: string,
  role?: string,
  tone?: string,
  brandVoice?: { defaultTone?: string; vocabularyRules?: any;
formattingRules?: string } | null,
): string {
  let prompt = ACTION_PROMPTS[action] || ACTION_PROMPTS.custo
m;

  if (role && role !== "general" && ROLE_PROMPTS[role]) {
    prompt += `\n\nWriting perspective: ${ROLE_PROMPTS[role]}
`;
  }

  if (tone) {
    prompt += `\n\nTone: Write in a ${tone} tone.`;
  }

  if (brandVoice) {
    if (brandVoice.defaultTone && !tone) {
      prompt += `\n\nDefault tone: ${brandVoice.defaultTone}
`;
    }
    if (brandVoice.vocabularyRules) {
      const rules = brandVoice.vocabularyRules as Array<{ us
e: string; notUse: string }>;
```

```
      if (rules.length > 0) {
        prompt += `\n\nVocabulary rules: ${rules.map(r => `Sa
y "${r.use}" instead of "${r.notUse}"`).join(". ")}`;
      }
    }
    if (brandVoice.formattingRules) {
      prompt += `\n\nFormatting rules: ${brandVoice.formattin
gRules}`;
    }
  }

  return prompt;
}

function buildUserPrompt(action: string, inputText?: string,
context?: string, customPrompt?: string, language?: string):
string {
  switch (action) {
    case "write":
      return customPrompt || "Write about this topic.";
    case "improve":
    case "shorten":
    case "lengthen":
    case "grammar":
      return `Text to ${action}:\n\n${inputText}`;
    case "tone":
      return `Rewrite in the specified tone:\n\n${inputText}
`;
    case "translate":
      return `Translate to ${language || "Spanish"}:\n\n${inp
utText}`;
    case "summarize":
      return `Summarize this:\n\n${inputText || context}`;
    case "brainstorm":
      return `Brainstorm ideas about: ${customPrompt || input
Text}`;
```

```
    case "outline":
      return `Create an outline for: ${customPrompt || inputT
ext}`;
    case "continue":
      return `Continue writing from here:\n\n${context?.slice
(-2000) || inputText}`;
    case "explain":
      return `Explain this:\n\n${inputText}`;
    case "action_items":
      return `Extract action items from:\n\n${inputText || co
ntext}`;
    case "custom":
      return `${customPrompt}\n\n${inputText ? `Text:\n${inpu
tText}` : ""}${context ? `\nContext:\n${context.slice(-3000)}
` : ""}`;
    default:
      return customPrompt || inputText || "";
  }
}

// ============================================================
==
// tRPC ROUTER
// ============================================================
==

export const aiWritingRouter = router({
  /**
   * Generate AI writing (streaming via SSE endpoint — see AP
I route below)
   * This non-streaming version is for programmatic use (agen
ts, etc.)
   */
  generate: protectedProcedure
    .input(z.object({
      action: writeActionSchema,
```

```
      inputText: z.string().optional(),
      context: z.string().optional(),
      customPrompt: z.string().optional(),
      tone: toneSchema.optional(),
      writerRole: writerRoleSchema.optional(),
      language: z.string().optional(),
      model: z.string().optional(),
      workspaceId: z.string().uuid(),
      pageId: z.string().uuid().optional(),
      channelId: z.string().uuid().optional(),
    }))
    .mutation(async ({ input, ctx }) => {
      const startTime = Date.now();

      // Load brand voice for workspace
      const brandVoice = await prisma.brandVoice.findUnique({
        where: { workspaceId: input.workspaceId },
      });

      const systemPrompt = buildSystemPrompt(
        input.action,
        input.writerRole,
        input.tone,
        brandVoice,
      );

      const userPrompt = buildUserPrompt(
        input.action,
        input.inputText,
        input.context,
        input.customPrompt,
        input.language,
      );

      const maxTokensMap: Record<string, number> = {
        summarize: 500, brainstorm: 1000, grammar: 1500,
```

```
        shorten: 1000, action_items: 800, write: 2000,
        outline: 1500, continue: 1500, improve: 2000,
        lengthen: 3000, translate: 2000, explain: 1000,
        tone: 2000, custom: 2000,
      };

      const temperatureMap: Record<string, number> = {
        grammar: 0.1, translate: 0.2, summarize: 0.3,
        action_items: 0.2, improve: 0.4, shorten: 0.3,
        lengthen: 0.5, tone: 0.4, write: 0.7,
        brainstorm: 0.9, outline: 0.5, continue: 0.7,
        explain: 0.4, custom: 0.7,
      };

      const result = await aiService.generateText(userPrompt,
{
        systemPrompt,
        model: (input.model as any) || undefined,
        maxTokens: maxTokensMap[input.action] || 2000,
        temperature: temperatureMap[input.action] || 0.7,
      });

      // Save to history
      const history = await prisma.aiWritingHistory.create({
        data: {
          userId: ctx.user.id,
          workspaceId: input.workspaceId,
          pageId: input.pageId,
          channelId: input.channelId,
          action: input.action,
          inputText: input.inputText,
          prompt: userPrompt,
          outputText: result.text,
          model: input.model || "gpt-4o",
          tone: input.tone,
          language: input.language,
```

```
          writerRole: input.writerRole,
          tokensUsed: result.tokensUsed.input + result.tokens
Used.output,
        },
      });

      // Log usage
      await logAiUsage({
        userId: ctx.user.id,
        workspaceId: input.workspaceId,
        feature: "writing",
        action: input.action,
        model: input.model || "gpt-4o",
        inputTokens: result.tokensUsed.input,
        outputTokens: result.tokensUsed.output,
        durationMs: Date.now() - startTime,
      });

      return {
        id: history.id,
        text: result.text,
        tokensUsed: result.tokensUsed,
      };
    }),

  /**
   * Mark a writing result as accepted (user clicked "Keep")
   */
  acceptResult: protectedProcedure
    .input(z.object({ id: z.string().uuid() }))
    .mutation(async ({ input }) => {
      await prisma.aiWritingHistory.update({
        where: { id: input.id },
        data: { accepted: true },
      });
    }),
```

```
  /**
   * Rate a writing result (1-5 stars)
   */
  rateResult: protectedProcedure
    .input(z.object({ id: z.string().uuid(), rating: z.number
().min(1).max(5) }))
    .mutation(async ({ input }) => {
      await prisma.aiWritingHistory.update({
        where: { id: input.id },
        data: { feedbackRating: input.rating },
      });
    }),

  /**
   * Get writing history for a user
   */
  getHistory: protectedProcedure
    .input(z.object({
      workspaceId: z.string().uuid(),
      limit: z.number().min(1).max(50).default(20),
      cursor: z.string().uuid().optional(),
    }))
    .query(async ({ input, ctx }) => {
      const items = await prisma.aiWritingHistory.findMany({
        where: { userId: ctx.user.id, workspaceId: input.work
spaceId },
        orderBy: { createdAt: "desc" },
        take: input.limit + 1,
        ...(input.cursor ? { cursor: { id: input.cursor }, sk
ip: 1 } : {}),
      });

      const hasMore = items.length > input.limit;
      return {
        items: items.slice(0, input.limit),
```

```
        nextCursor: hasMore ? items[input.limit - 1].id : und
efined,
      };
    }),

  /**
   * Save/update brand voice settings
   */
  saveBrandVoice: protectedProcedure
    .input(z.object({
      workspaceId: z.string().uuid(),
      defaultTone: z.string().optional(),
      styleGuideText: z.string().optional(),
      vocabularyRules: z.array(z.object({ use: z.string(), no
tUse: z.string() })).optional(),
      formattingRules: z.string().optional(),
      customRoles: z.array(z.object({ name: z.string(), syste
mPrompt: z.string() })).optional(),
    }))
    .mutation(async ({ input }) => {
      return prisma.brandVoice.upsert({
        where: { workspaceId: input.workspaceId },
        create: {
          workspaceId: input.workspaceId,
          defaultTone: input.defaultTone || "professional",
          styleGuideText: input.styleGuideText,
          vocabularyRules: input.vocabularyRules,
          formattingRules: input.formattingRules,
          customRoles: input.customRoles,
        },
        update: {
          defaultTone: input.defaultTone,
          styleGuideText: input.styleGuideText,
          vocabularyRules: input.vocabularyRules,
          formattingRules: input.formattingRules,
          customRoles: input.customRoles,
```

```
        },
      });
    }),

  /**
   * Get brand voice settings
   */
  getBrandVoice: protectedProcedure
    .input(z.object({ workspaceId: z.string().uuid() }))
    .query(async ({ input }) => {
      return prisma.brandVoice.findUnique({
        where: { workspaceId: input.workspaceId },
      });
    }),
});
```

1.2 Streaming API Route (for real-time text generation)

Create file: `src/app/api/ai/writing/stream/route.ts`

```
// src/app/api/ai/writing/stream/route.ts
// Server-Sent Events endpoint for streaming AI writing respo
nses

import { NextRequest, NextResponse } from "next/server";
import { getServerSession } from "next-auth";
import { authOptions } from "@/lib/auth";
import { aiService } from "@/lib/ai/ai-service";
import { prisma } from "@/lib/prisma";

export async function POST(req: NextRequest) {
  const session = await getServerSession(authOptions);
  if (!session?.user?.id) {
    return NextResponse.json({ error: "Unauthorized" }, { sta
tus: 401 });
  }
```

```
  const body = await req.json();
  const { action, inputText, context, customPrompt, tone, wri
terRole, language, model, workspaceId } = body;

  // Load brand voice
  const brandVoice = await prisma.brandVoice.findUnique({
    where: { workspaceId },
  });

  // Build prompts (reuse same logic from router)
  const ACTION_PROMPTS: Record<string, string> = {
    write: "You are an expert writer. Generate clear, well-st
ructured content based on the user's request.",
    improve: "You are an expert editor. Improve the following
text to be clearer and better structured.",
    shorten: "Condense the following text to be shorter while
keeping all key information.",
    lengthen: "Expand the following text with more detail and
examples.",
    grammar: "Fix all spelling, grammar, and punctuation erro
rs. Make minimal changes.",
    translate: "Translate accurately into the specified langu
age. Preserve formatting and tone.",
    summarize: "Create a concise summary in 2-4 sentences.",
    brainstorm: "Generate 5-10 unique, practical ideas.",
    outline: "Create a clear, hierarchical outline with headi
ngs and bullet points.",
    continue: "Continue writing from where the text ends. Mat
ch the existing style.",
    explain: "Provide a clear, easy-to-understand explanatio
n.",
    action_items: "Extract all action items as clear, actiona
ble checkbox items.",
    custom: "You are an expert AI assistant.",
  };
```

```
  let systemPrompt = ACTION_PROMPTS[action] || ACTION_PROMPT
S.custom;
  if (tone) systemPrompt += ` Write in a ${tone} tone.`;
  if (writerRole && writerRole !== "general") systemPrompt +=
` Writing as a ${writerRole}.`;

  let userPrompt = customPrompt || inputText || "";
  if (action === "improve" && inputText) userPrompt = `Improv
e this text:\n\n${inputText}`;
  if (action === "translate") userPrompt = `Translate to ${la
nguage}:\n\n${inputText}`;
  if (action === "continue" && context) userPrompt = `Continu
e from:\n\n${context.slice(-2000)}`;

  // Stream response using ReadableStream
  const encoder = new TextEncoder();
  const stream = new ReadableStream({
    async start(controller) {
      try {
        const generator = aiService.generateStream(userPromp
t, {
          systemPrompt,
          model: model || undefined,
          maxTokens: 2000,
          temperature: 0.7,
        });

        for await (const chunk of generator) {
          if (chunk.text) {
            controller.enqueue(encoder.encode(chunk.text));
          }
          if (chunk.done) {
            controller.close();
            return;
          }
```

```
        }
        controller.close();
      } catch (error) {
        controller.enqueue(encoder.encode("\n\n[Error: AI gen
eration failed. Please try again.]"));
        controller.close();
      }
    },
  });

  return new NextResponse(stream, {
    headers: {
      "Content-Type": "text/plain; charset=utf-8",
      "Cache-Control": "no-cache",
      "Transfer-Encoding": "chunked",
    },
  });
}
```

1.3 Writing Assistant UI — Editor Integration

Create file: `src/components/ai/ai-writing-toolbar.tsx`

```
// src/components/ai/ai-writing-toolbar.tsx
// Floating toolbar that appears when text is selected in the
editor

"use client";

import { useState } from "react";
import { Sparkles, ChevronDown } from "lucide-react";
import {
  DropdownMenu, DropdownMenuTrigger, DropdownMenuContent,
  DropdownMenuItem, DropdownMenuSub, DropdownMenuSubTrigger,
  DropdownMenuSubContent, DropdownMenuSeparator,
} from "@/components/ui/dropdown-menu";
```

```
interface AiWritingToolbarProps {
  selectedText: string;
  onAction: (action: string, options?: { tone?: string; langu
age?: string; customPrompt?: string }) => void;
}

const LANGUAGES = [
  "Spanish", "Hindi", "French", "German", "Japanese", "Chines
e",
  "Portuguese", "Arabic", "Korean", "Russian", "Italian", "Du
tch",
  "Turkish", "Vietnamese", "Thai", "Polish", "Swedish", "Indo
nesian",
  "Bengali", "Tamil", "Telugu", "Marathi",
];

const TONES = [
  { value: "professional", label: "Professional" },
  { value: "casual", label: "Casual" },
  { value: "friendly", label: "Friendly" },
  { value: "academic", label: "Academic" },
  { value: "persuasive", label: "Persuasive" },
];

export function AiWritingToolbar({ selectedText, onAction }:
AiWritingToolbarProps) {
  const [customPrompt, setCustomPrompt] = useState("");
  const [showCustomInput, setShowCustomInput] = useState(fals
e);

  if (!selectedText) return null;

  return (
    <DropdownMenu>
      <DropdownMenuTrigger asChild>
```

```
        <button className="flex items-center gap-1 px-2 py-1
text-sm bg-purple-600 text-white rounded hover:bg-purple-70
0">
          <Sparkles className="w-3 h-3" />
          AI
          <ChevronDown className="w-3 h-3" />
        </button>
      </DropdownMenuTrigger>
      <DropdownMenuContent align="start" className="w-56">
        <DropdownMenuItem onClick={() => onAction("improve")}
>
          ✨ Improve writing
        </DropdownMenuItem>
        <DropdownMenuItem onClick={() => onAction("grammar")}
>
          📝 Fix spelling & grammar
        </DropdownMenuItem>
        <DropdownMenuItem onClick={() => onAction("shorten")}
>
          📐 Make shorter
        </DropdownMenuItem>
        <DropdownMenuItem onClick={() => onAction("lengthe
n")}>
          📏 Make longer
        </DropdownMenuItem>
        <DropdownMenuSeparator />
        <DropdownMenuSub>
          <DropdownMenuSubTrigger>🎭 Change tone</DropdownMen
uSubTrigger>
          <DropdownMenuSubContent>
            {TONES.map((t) => (
              <DropdownMenuItem key={t.value} onClick={() =>
onAction("tone", { tone: t.value })}>
                {t.label}
              </DropdownMenuItem>
            ))}
```

```
          </DropdownMenuSubContent>
        </DropdownMenuSub>
        <DropdownMenuItem onClick={() => onAction("explain")}
>
          💡 Simplify language
        </DropdownMenuItem>
        <DropdownMenuSeparator />
        <DropdownMenuItem onClick={() => onAction("summariz
e")}>
          📋 Summarize
        </DropdownMenuItem>
        <DropdownMenuItem onClick={() => onAction("explain")}
>
          🔍 Explain this
        </DropdownMenuItem>
        <DropdownMenuSub>
          <DropdownMenuSubTrigger>🌍 Translate</DropdownMenuS
ubTrigger>
          <DropdownMenuSubContent className="max-h-[300px] ov
erflow-y-auto">
            {LANGUAGES.map((lang) => (
              <DropdownMenuItem key={lang} onClick={() => onA
ction("translate", { language: lang })}>
                {lang}
              </DropdownMenuItem>
            ))}
          </DropdownMenuSubContent>
        </DropdownMenuSub>
        <DropdownMenuItem onClick={() => onAction("action_ite
ms")}>
          ☑ Extract action items
        </DropdownMenuItem>
        <DropdownMenuSeparator />
        <DropdownMenuItem onClick={() => setShowCustomInput(t
rue)}>
          ✏ Custom instruction...
```

```
        </DropdownMenuItem>
      </DropdownMenuContent>

      {/* Custom prompt modal */}
      {showCustomInput && (
        <div className="fixed inset-0 bg-black/50 flex items-
center justify-center z-50">
          <div className="bg-white rounded-lg p-4 w-96 shadow
-xl">
            <h3 className="text-sm font-medium mb-2">Custom A
I instruction</h3>
            <textarea
              value={customPrompt}
              onChange={(e) => setCustomPrompt(e.target.valu
e)}
              placeholder="Tell AI what to do with the select
ed text..."
              className="w-full border rounded p-2 text-sm h-
24 resize-none focus:outline-none focus:ring-2 focus:ring-pur
ple-500"
              autoFocus
            />
            <div className="flex justify-end gap-2 mt-2">
              <button
                onClick={() => setShowCustomInput(false)}
                className="text-sm px-3 py-1 text-gray-600 ho
ver:text-gray-800"
              >
                Cancel
              </button>
              <button
                onClick={() => {
                  onAction("custom", { customPrompt });
                  setShowCustomInput(false);
                  setCustomPrompt("");
                }}
```

```
                  className="text-sm px-3 py-1 bg-purple-600 te
xt-white rounded hover:bg-purple-700"
                >
                  Run AI
                </button>
              </div>
            </div>
          </div>
        )}
      </DropdownMenu>
    );
}
```

1.4 Slash Command AI Menu

Create file: `src/components/ai/ai-slash-menu.tsx`

```
// src/components/ai/ai-slash-menu.tsx
// Shows AI options when user types "/" in editor

"use client";

import { useState } from "react";
import { Sparkles, FileText, Lightbulb, List, ArrowRight, Lan
guages, CheckSquare, Table, Code } from "lucide-react";

interface SlashMenuItem {
  id: string;
  label: string;
  description: string;
  icon: React.ReactNode;
  action: string;
}

const AI_SLASH_ITEMS: SlashMenuItem[] = [
  { id: "write", label: "AI Write", description: "Generate co
```

```
ntent from a prompt", icon: <Sparkles className="w-4 h-4" />,
action: "write" },
  { id: "brainstorm", label: "AI Brainstorm", description: "G
enerate 5-10 ideas", icon: <Lightbulb className="w-4 h-4" />,
action: "brainstorm" },
  { id: "outline", label: "AI Outline", description: "Create
structured outline", icon: <List className="w-4 h-4" />, acti
on: "outline" },
  { id: "continue", label: "AI Continue", description: "Conti
nue writing from here", icon: <ArrowRight className="w-4 h-4"
/>, action: "continue" },
  { id: "summarize", label: "AI Summarize", description: "Sum
marize this page", icon: <FileText className="w-4 h-4" />, ac
tion: "summarize" },
  { id: "translate", label: "AI Translate", description: "Tra
nslate page content", icon: <Languages className="w-4 h-4" /
>, action: "translate" },
  { id: "action_items", label: "AI Action Items", descriptio
n: "Extract tasks from page", icon: <CheckSquare className="w
-4 h-4" />, action: "action_items" },
  { id: "table", label: "AI Table", description: "Generate a
data table", icon: <Table className="w-4 h-4" />, action: "wr
ite" },
  { id: "code", label: "AI Code", description: "Generate code
block", icon: <Code className="w-4 h-4" />, action: "write"
},
];

interface AiSlashMenuProps {
  filter: string; // what the user typed after "/"
  onSelect: (action: string, label: string) => void;
  onClose: () => void;
}

export function AiSlashMenu({ filter, onSelect, onClose }: Ai
SlashMenuProps) {
```

```
  const filtered = AI_SLASH_ITEMS.filter(
    (item) =>
      item.label.toLowerCase().includes(filter.toLowerCase())
||
      item.description.toLowerCase().includes(filter.toLowerC
ase())
  );

  const [selectedIndex, setSelectedIndex] = useState(0);

  if (filtered.length === 0) return null;

  return (
    <div className="absolute z-50 bg-white border rounded-lg
shadow-lg w-72 max-h-80 overflow-y-auto">
      <div className="p-1.5 text-xs text-gray-500 font-medium
border-b">AI Actions</div>
      {filtered.map((item, index) => (
        <button
          key={item.id}
          onClick={() => onSelect(item.action, item.label)}
          className={`w-full flex items-center gap-3 px-3 py-
2 text-left hover:bg-gray-100 ${
            index === selectedIndex ? "bg-gray-100" : ""
          }`}
        >
          <span className="text-purple-600">{item.icon}</span
>
          <div>
            <div className="text-sm font-medium">{item.label}
</div>
            <div className="text-xs text-gray-500">{item.desc
ription}</div>
          </div>
        </button>
      ))}
```

```
    </div>
  );
}
```

FEATURE 2: AI KNOWLEDGE SEARCH

2.1 tRPC Router

Create file: `src/server/routers/knowledge.ts`

```
// src/server/routers/knowledge.ts

import { z } from "zod";
import { router, protectedProcedure } from "@/lib/trpc";
import { prisma } from "@/lib/prisma";
import { aiService } from "@/lib/ai/ai-service";
import { getEmbedding } from "@/lib/ai/embeddings";
import { logAiUsage } from "@/server/services/ai-usage-logge
r";

export const knowledgeRouter = router({
  /**
   * Keyword search — instant results as user types (for Sear
ch tab)
   */
  search: protectedProcedure
    .input(z.object({
      workspaceId: z.string().uuid(),
      query: z.string().min(1).max(500),
      sourceType: z.string().optional(), // filter by type
      limit: z.number().min(1).max(50).default(20),
    }))
    .query(async ({ input, ctx }) => {
      // Full-text search using PostgreSQL ts_vector
      const results = await prisma.$queryRaw<Array<{
```

```
        id: string;
        sourceType: string;
        sourceId: string;
        title: string;
        content: string;
        metadata: any;
        accessLevel: string;
        rank: number;
      }>>`
        SELECT
          id, "sourceType", "sourceId", title,
          ts_headline('english', content, plainto_tsquery('en
glish', ${input.query}),
            'StartSel=<mark>, StopSel=</mark>, MaxFragments=
2, MaxWords=40') as content,
          metadata, "accessLevel",
          ts_rank(to_tsvector('english', content), plainto_ts
query('english', ${input.query})) as rank
        FROM "KnowledgeIndex"
        WHERE "workspaceId" = ${input.workspaceId}::uuid
          AND to_tsvector('english', content) @@ plainto_tsqu
ery('english', ${input.query})
          ${input.sourceType ? prisma.$queryRaw`AND "sourceTy
pe" = ${input.sourceType}` : prisma.$queryRaw``}
        ORDER BY rank DESC
        LIMIT ${input.limit}
      `;

      // TODO: Add permission filtering based on accessLevel
and user's channel memberships

      // Group results by sourceType
      const grouped: Record<string, typeof results> = {};
      for (const r of results) {
        if (!grouped[r.sourceType]) grouped[r.sourceType] =
[];
```

```
        grouped[r.sourceType].push(r);
      }

      return { results, grouped, totalCount: results.length
};
    }),

  /**
   * AI Q&A — natural language question → AI-generated answer
with citations
   */
  askAi: protectedProcedure
    .input(z.object({
      workspaceId: z.string().uuid(),
      query: z.string().min(1).max(1000),
      model: z.string().optional(),
    }))
    .mutation(async ({ input, ctx }) => {
      const startTime = Date.now();

      // Step 1: Generate embedding for the question
      const queryEmbedding = await getEmbedding(input.query);

      // Step 2: Semantic search — find most relevant content
      const semanticResults = await prisma.$queryRaw<Array<{
        id: string;
        sourceType: string;
        sourceId: string;
        title: string;
        content: string;
        metadata: any;
        similarity: number;
      }>>`
        SELECT
          id, "sourceType", "sourceId", title,
          LEFT(content, 1000) as content,
```

```
          metadata,
          1 - (embedding <=> ${queryEmbedding}::vector) as si
milarity
        FROM "KnowledgeIndex"
        WHERE "workspaceId" = ${input.workspaceId}::uuid
          AND embedding IS NOT NULL
        ORDER BY embedding <=> ${queryEmbedding}::vector
        LIMIT 10
      `;

      // Step 3: Also do keyword search for hybrid results
      const keywordResults = await prisma.$queryRaw<Array<{
        id: string;
        sourceType: string;
        sourceId: string;
        title: string;
        content: string;
        metadata: any;
      }>>`
        SELECT id, "sourceType", "sourceId", title, LEFT(cont
ent, 1000) as content, metadata
        FROM "KnowledgeIndex"
        WHERE "workspaceId" = ${input.workspaceId}::uuid
          AND to_tsvector('english', content) @@ plainto_tsqu
ery('english', ${input.query})
        ORDER BY ts_rank(to_tsvector('english', content), pla
into_tsquery('english', ${input.query})) DESC
        LIMIT 5
      `;

      // Step 4: Merge and deduplicate results
      const allResults = [...semanticResults];
      for (const kr of keywordResults) {
        if (!allResults.find((r) => r.sourceId === kr.sourceI
d)) {
          allResults.push({ ...kr, similarity: 0.5 });
```

```
        }
      }

      // Step 5: Build context for AI from top results
      const contextForAI = allResults
        .slice(0, 8)
        .map((r, i) => `[Source ${i + 1}: ${r.sourceType}
"${r.title}"]\n${r.content}`)
        .join("\n\n---\n\n");

      // Step 6: Generate AI answer with citations
      const result = await aiService.generateText(
        `Based on the following workspace data, answer the us
er's question accurately.
Include citations in your answer like [Source 1], [Source 2]
when you reference specific information.
If the information is not found in the sources, say "I could
n't find information about that in your workspace."

USER QUESTION: ${input.query}

WORKSPACE DATA:
${contextForAI}`,
        {
          systemPrompt: "You are an AI assistant that answers
questions based on workspace data. Always cite your sources.
Be concise and accurate. If you're not sure, say so.",
          model: (input.model as any) || "gpt-4o",
          maxTokens: 1000,
          temperature: 0.3,
        }
      );

      // Step 7: Build citations array
      const citations = allResults.slice(0, 8).map((r) => ({
        sourceId: r.sourceId,
```

```
        sourceType: r.sourceType,
        title: r.title,
        snippet: r.content.slice(0, 200),
        metadata: r.metadata,
      }));

      // Step 8: Save query for analytics
      await prisma.knowledgeQuery.create({
        data: {
          userId: ctx.user.id,
          workspaceId: input.workspaceId,
          query: input.query,
          results: allResults.slice(0, 8).map((r) => ({
            sourceType: r.sourceType,
            sourceId: r.sourceId,
            title: r.title,
            relevanceScore: r.similarity,
          })),
          aiAnswer: result.text,
          citations,
          responseTimeMs: Date.now() - startTime,
        },
      });

      // Log usage
      await logAiUsage({
        userId: ctx.user.id,
        workspaceId: input.workspaceId,
        feature: "search",
        action: "ask_ai",
        model: input.model || "gpt-4o",
        inputTokens: result.tokensUsed.input,
        outputTokens: result.tokensUsed.output,
        durationMs: Date.now() - startTime,
      });
```

```
      return {
        answer: result.text,
        citations,
        responseTimeMs: Date.now() - startTime,
      };
    }),

  /**
   * Rate a search/AI answer
   */
  rateFeedback: protectedProcedure
    .input(z.object({ queryId: z.string().uuid(), rating: z.n
umber().min(1).max(5) }))
    .mutation(async ({ input }) => {
      await prisma.knowledgeQuery.update({
        where: { id: input.queryId },
        data: { feedbackRating: input.rating },
      });
    }),
});
```

2.2 Universal Search UI

Create file: `src/components/search/universal-search.tsx`

```
// src/components/search/universal-search.tsx
// Universal search modal — Ctrl+K / Cmd+K

"use client";

import { useState, useEffect, useRef, useCallback } from "rea
ct";
import { Search, Sparkles, X, FileText, CheckSquare, MessageC
ircle, Paperclip, Loader2, ThumbsUp, ThumbsDown } from "lucid
e-react";
import { AiStreamingDisplay } from "@/components/ai/ai-stream
```

```
ing-display";
import { trpc } from "@/lib/trpc-client";
import { useDebounce } from "@/hooks/use-debounce";

interface UniversalSearchProps {
  workspaceId: string;
  isOpen: boolean;
  onClose: () => void;
}

type Tab = "search" | "ask_ai";

const SOURCE_ICONS: Record<string, React.ReactNode> = {
  page: <FileText className="w-4 h-4 text-blue-500" />,
  task: <CheckSquare className="w-4 h-4 text-green-500" />,
  message: <MessageCircle className="w-4 h-4 text-purple-500"
/>,
  meeting_transcript: <FileText className="w-4 h-4 text-orang
e-500" />,
  file: <Paperclip className="w-4 h-4 text-gray-500" />,
};

export function UniversalSearch({ workspaceId, isOpen, onClos
e }: UniversalSearchProps) {
  const [tab, setTab] = useState<Tab>("search");
  const [query, setQuery] = useState("");
  const debouncedQuery = useDebounce(query, 200);
  const inputRef = useRef<HTMLInputElement>(null);

  // Keyword search
  const searchQuery = trpc.knowledge.search.useQuery(
    { workspaceId, query: debouncedQuery },
    { enabled: tab === "search" && debouncedQuery.length > 0
}
  );
```

```
  // AI Q&A
  const askAiMutation = trpc.knowledge.askAi.useMutation();

  // Focus input on open
  useEffect(() => {
    if (isOpen) {
      setTimeout(() => inputRef.current?.focus(), 100);
    }
  }, [isOpen]);

  // Keyboard shortcut: Cmd+K / Ctrl+K
  useEffect(() => {
    const handler = (e: KeyboardEvent) => {
      if ((e.metaKey || e.ctrlKey) && e.key === "k") {
        e.preventDefault();
        if (isOpen) onClose();
        // Parent component should handle opening
      }
      if (e.key === "Escape" && isOpen) onClose();
    };
    document.addEventListener("keydown", handler);
    return () => document.removeEventListener("keydown", hand
ler);
  }, [isOpen, onClose]);

  const handleAskAi = useCallback(() => {
    if (!query.trim()) return;
    askAiMutation.mutate({ workspaceId, query });
  }, [query, workspaceId, askAiMutation]);

  if (!isOpen) return null;

  return (
    <div className="fixed inset-0 z-50 flex items-start justi
fy-center pt-[15vh] bg-black/50" onClick={onClose}>
      <div
```

```
        className="bg-white rounded-xl shadow-2xl w-full max-
w-2xl max-h-[65vh] flex flex-col"
        onClick={(e) => e.stopPropagation()}
      >
        {/* Tabs */}
        <div className="flex border-b">
          <button
            onClick={() => setTab("search")}
            className={`flex items-center gap-2 px-4 py-3 tex
t-sm font-medium border-b-2 ${
              tab === "search" ? "border-blue-600 text-blue-6
00" : "border-transparent text-gray-500"
            }`}
          >
            <Search className="w-4 h-4" /> Search
          </button>
          <button
            onClick={() => setTab("ask_ai")}
            className={`flex items-center gap-2 px-4 py-3 tex
t-sm font-medium border-b-2 ${
              tab === "ask_ai" ? "border-purple-600 text-purp
le-600" : "border-transparent text-gray-500"
            }`}
          >
            <Sparkles className="w-4 h-4" /> Ask AI
          </button>
          <div className="flex-1" />
          <button onClick={onClose} className="px-3 text-gray
-400 hover:text-gray-600">
            <X className="w-4 h-4" />
          </button>
        </div>

        {/* Search input */}
        <div className="px-4 py-3 border-b">
          <div className="flex items-center gap-2">
```

```
            {tab === "search" ? <Search className="w-4 h-4 te
xt-gray-400" /> : <Sparkles className="w-4 h-4 text-purple-40
0" />}
            <input
              ref={inputRef}
              value={query}
              onChange={(e) => setQuery(e.target.value)}
              onKeyDown={(e) => {
                if (e.key === "Enter" && tab === "ask_ai") {
                  e.preventDefault();
                  handleAskAi();
                }
              }}
              placeholder={tab === "search" ? "Search pages,
tasks, messages..." : "Ask a question about your workspac
e..."}
              className="flex-1 text-sm outline-none"
            />
            {tab === "ask_ai" && (
              <button
                onClick={handleAskAi}
                disabled={askAiMutation.isPending || !query.t
rim()}
                className="text-sm px-3 py-1 bg-purple-600 te
xt-white rounded hover:bg-purple-700 disabled:opacity-50"
              >
                Ask
              </button>
            )}
          </div>
        </div>

        {/* Results */}
        <div className="flex-1 overflow-y-auto p-4">
          {tab === "search" && (
            <>
```

```
              {searchQuery.isLoading && <div className="text-
sm text-gray-500 text-center py-8">Searching...</div>}
              {searchQuery.data && searchQuery.data.totalCoun
t === 0 && (
                <div className="text-sm text-gray-500 text-ce
nter py-8">No results found for "{query}"</div>
              )}
              {searchQuery.data && Object.entries(searchQuer
y.data.grouped).map(([type, items]) => (
                <div key={type} className="mb-4">
                  <h3 className="text-xs font-medium text-gra
y-500 uppercase mb-2 flex items-center gap-1">
                    {SOURCE_ICONS[type]} {type}s ({(items as
any[]).length})
                  </h3>
                  {(items as any[]).map((item: any) => (
                    <button
                      key={item.id}
                      className="w-full text-left px-3 py-2 r
ounded hover:bg-gray-100 block"
                      onClick={() => {
                        // Navigate to source
                        onClose();
                      }}
                    >
                      <div className="text-sm font-medium tex
t-gray-800">{item.title}</div>
                      <div
                        className="text-xs text-gray-500 mt-
0.5 line-clamp-2"
                        dangerouslySetInnerHTML={{ __html: it
em.content }}
                      />
                    </button>
                  ))}
                </div>
```

```
              ))}
            </>
          )}

          {tab === "ask_ai" && (
            <>
              {askAiMutation.isPending && (
                <div className="flex items-center gap-2 text-
sm text-purple-600 py-8 justify-center">
                  <Sparkles className="w-4 h-4" />
                  <span>Searching workspace and generating an
swer...</span>
                  <Loader2 className="w-4 h-4 animate-spin" /
>
                </div>
              )}
              {askAiMutation.data && (
                <div>
                  <div className="flex items-center gap-1 tex
t-xs text-purple-600 mb-2">
                    <Sparkles className="w-3 h-3" /> AI Answe
r
                  </div>
                  <div className="text-sm text-gray-800 prose
prose-sm mb-4 whitespace-pre-wrap">
                    {askAiMutation.data.answer}
                  </div>
                  {askAiMutation.data.citations.length > 0 &&
(
                    <div>
                      <div className="text-xs font-medium tex
t-gray-500 mb-1">Sources:</div>
                      {askAiMutation.data.citations.map((c: a
ny, i: number) => (
                        <button
                          key={i}
```

```
                          className="flex items-center gap-2
text-xs text-blue-600 hover:underline py-0.5"
                        >
                          {SOURCE_ICONS[c.sourceType]}
                          {c.title}
                        </button>
                      ))}
                    </div>
                  )}
                  <div className="flex items-center gap-2 mt-
3 pt-3 border-t">
                    <span className="text-xs text-gray-400">H
elpful?</span>
                    <button className="text-gray-400 hover:te
xt-green-600"><ThumbsUp className="w-3 h-3" /></button>
                    <button className="text-gray-400 hover:te
xt-red-600"><ThumbsDown className="w-3 h-3" /></button>
                    <span className="text-xs text-gray-400 ml
-2">
                      {askAiMutation.data.responseTimeMs}ms
                    </span>
                  </div>
                </div>
              )}
              {!askAiMutation.data && !askAiMutation.isPendin
g && (
                <div className="text-sm text-gray-500 text-ce
nter py-8">
                  Ask any question about your workspace data.
                  <br />
                  <span className="text-xs text-gray-400">
                    "What was decided about pricing?" · "Who
is working on the auth feature?" · "Summarize last week's mee
tings"
                  </span>
                </div>
```

```
              )}
            </>
          )}
        </div>
      </div>
    </div>
  );
}
```

EDGE CASES FOR SPRINT 2

```
FEATURE 1 (Writing):
1. Empty selection + "Improve" action → Show toast: "Select s
ome text first"
2. Selected text > 10,000 chars → Truncate to last 10k chars,
show warning
3. AI returns empty response → Show "AI couldn't generate con
tent. Try rephrasing."
4. User clicks "Try Again" → Re-run with temperature + 0.1 (m
ax 3 retries)
5. Network failure during streaming → Save partial text, show
[Keep Partial] [Retry]
6. Concurrent AI requests → Queue them, show "Processing prev
ious request..."
7. Brand voice not set → Silently skip brand voice in prompt
(no error)
8. Unknown action type → Fallback to "custom" action

FEATURE 2 (Knowledge Search):
1. No results found → Show "No results" with suggestion to tr
y different keywords
2. Empty knowledge index (new workspace) → Show "Your workspa
ce is empty. Add pages and tasks to make them searchable."
3. AI Q&A with no relevant results → AI says "I couldn't find
```

```
information about that in your workspace"
4. Very long AI answer → Cap at 1000 tokens, show full answer
in expandable section
5. Search query < 2 chars → Don't trigger search, show recent
searches instead
6. Indexing job fails → Mark entry as isStale, retry on next
indexing cycle
7. Embedding generation fails → Fall back to keyword-only sea
rch for that document
8. Permission filtering: Message from private channel → Only
return if user is channel member
```

SPRINT 2 VERIFICATION CHECKLIST

```
□ Open editor → press Space on empty line → AI input appears
□ Type "Write an introduction for our product" → AI streams t
ext into editor
□ Click [Keep] → text inserted into editor
□ Select text → click AI button → choose "Improve" → see diff
→ Accept
□ Type /ai brainstorm → generates ideas
□ Press Cmd+J → prompt bar opens → type question → get respon
se
□ AI writing with "Write as marketer" role → noticeably diffe
rent tone
□ Brand voice settings saved → AI follows vocabulary rules
□ Press Cmd+K → universal search opens → type keyword → resul
ts appear grouped by type
□ Switch to "Ask AI" tab → ask question → get answer with cit
ations
□ Click citation source → navigates to correct page/message/t
ask
□ Create a new page → within 5 minutes it appears in search r
esults
```

```
□ AI Q&A with no relevant data → shows "couldn't find" messag
e gracefully
```

Once all checks pass, proceed to Sprint 3.